Loop Article of Week

How WhatsApp Handled 1 Billion Users with 50 Engineers

The Erlang actor model, FreeBSD networking, and the protocol decisions behind the world's most efficient messaging system

In 2014, Facebook paid $19 billion for WhatsApp.

At the time, WhatsApp had **450** million users, was growing by 1 million users per day, and was processing 50 billion messages daily. The engineering team maintaining this infrastructure: **50 engineers**.

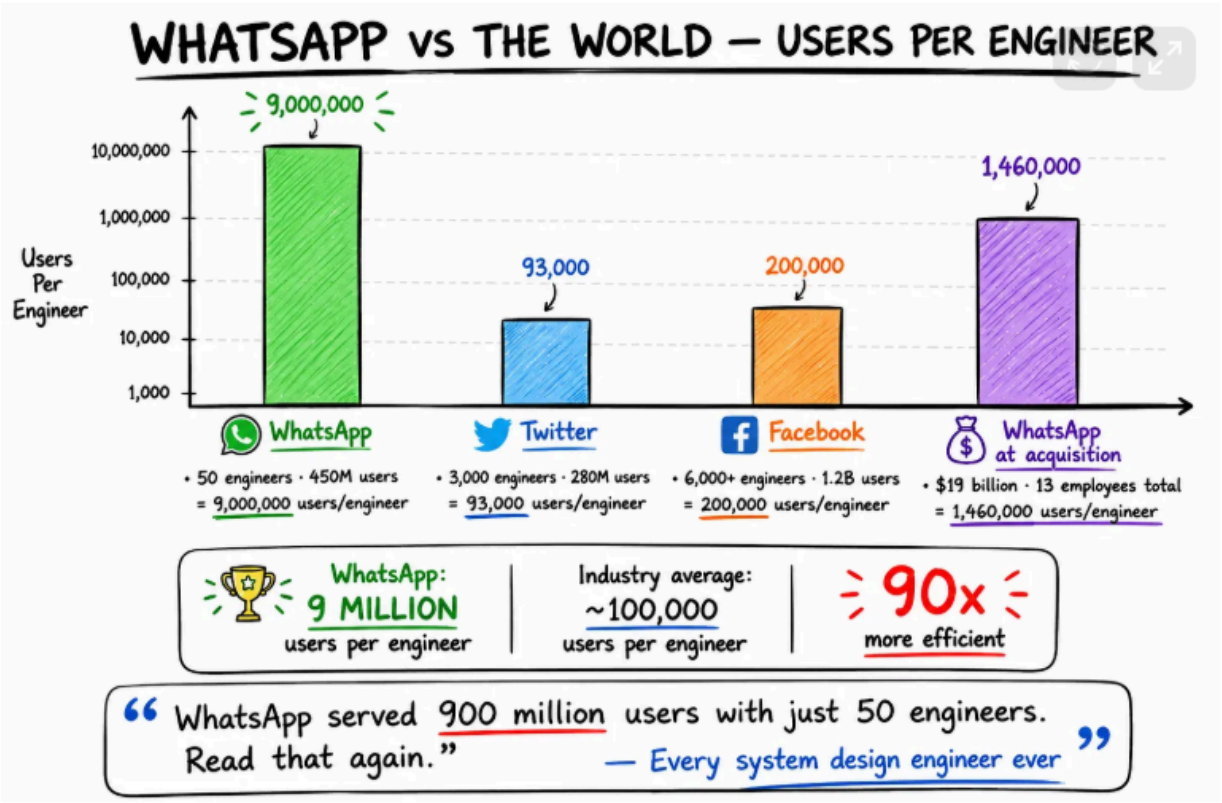
For comparison: Twitter had ~3,000 engineers at similar scale. Facebook itself had over 6,000.

WhatsApp's ratio of users per engineer was roughly 10 million to one ,the most efficient large-scale system ever built.

WhatsApp made a series of technical decisions that most engineers never learn about:

- They chose **Erlang** when everyone used Java
- They chose **FreeBSD** when everyone used Linux
- They built on **ejabberd** when everyone wanted to build from scratch
- They chose to do **one thing** when every competitor added features

Lets understand each decision made by WhatsAPP in this series , the **Erlang** actor model, **Mnesia**, **FreeBSD** kernel tuning, the custom **XMPP** protocol, message delivery flow, end-to-end encryption with Signal Protocol, and the engineering culture behind it all.



The Three Engineering Principles

Before any technology, WhatsApp had a philosophy. Three principles that governed every decision:

1. Keep it very simple. The codebase, the architecture, the product. Simplicity at every layer.

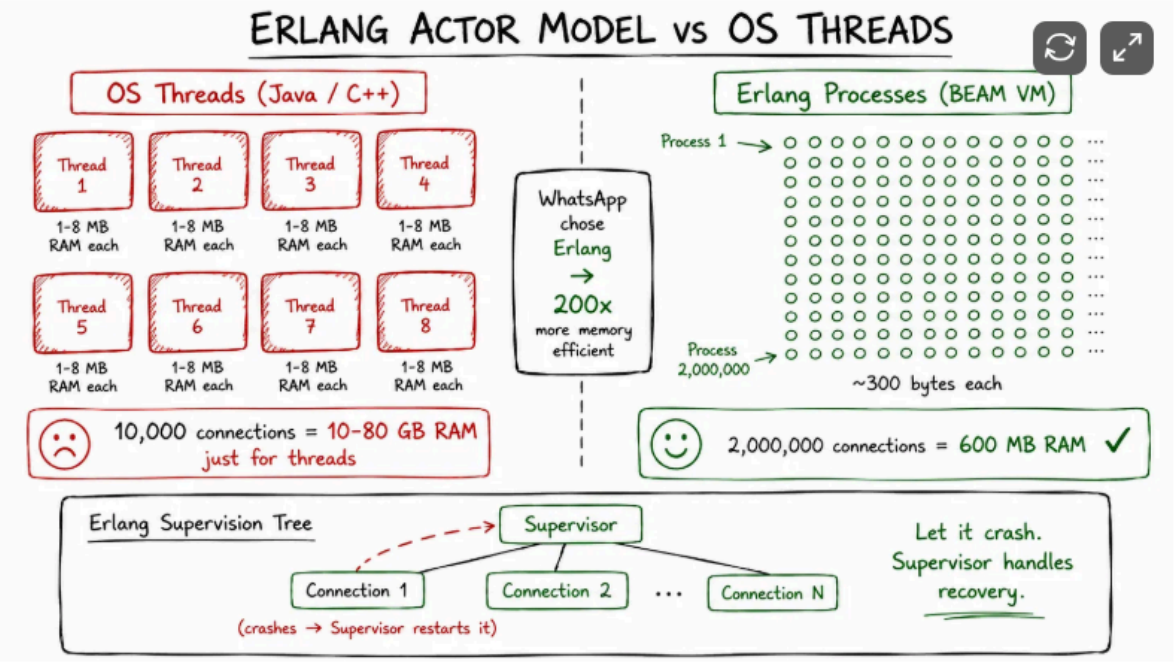
2. Don't reinvent the wheel. Build on open source. Use commercial solutions. Write only what gives you competitive advantage.

3. Use proven, solid technologies. Not the newest. Not the trendiest. Whatever works best for the specific problem.

These principles explain every technical choice that follows. Erlang isn't new. ejabberd isn't custom. FreeBSD isn't modern. But all three were exactly right for WhatsApp's specific problem: millions of persistent connections, low latency message delivery, high availability, tiny team.

Decision 1: Erlang - The Language Nobody Uses

WhatsApp's backend is built on **Erlang** running on the BEAM virtual machine, using a modified version of XMPP via Ejabberd for messaging and presence, and Mnesia as the primary distributed database. This stack enables WhatsApp to handle billions of concurrent connections with low latency.



Erlang was created in 1987 by Ericsson for telephone switching systems — networks that must handle millions of simultaneous calls and absolutely cannot go down. It's a functional language. Most engineers have never written a line of it. Its ecosystem is tiny compared to Java or Python.

WhatsApp chose it anyway. Here's why.

The actor model: processes as citizens

In Java or C++, handling concurrent connections means OS threads. Each thread costs 1–8MB of memory and requires the OS to save the entire CPU state on every context switch. At 10,000 concurrent connections, you're already using gigabytes of RAM just for thread overhead.

Erlang's model is different. Instead of OS threads, Erlang has **lightweight processes** — isolated units of computation managed entirely by the BEAM virtual machine. Each process costs roughly **300 bytes of memory**. Starting one takes microseconds. The BEAM scheduler handles switching between them with no OS involvement.

OS Thread model (Java/C++):
Each connection = 1 thread = 1-8MB RAM
10,000 connections = 10-80 GB RAM (just overhead)
Context switch = OS must save full CPU state

Erlang process model:
Each connection = 1 process = ~300 bytes RAM

2,000,000 connections = 600 MB RAM (just overhead)
Context switch = BEAM scheduler, microseconds

This is why WhatsApp scaled to 2 billion users with 50 engineers by using Erlang — lightweight processes handling 2M+ connections per server.

Each user is a process

In WhatsApp's architecture, every active user connection gets its own Erlang process. When Alice sends a message to Bob, the message passes directly from Alice's process to Bob's process via Erlang's message-passing primitives — no locks, no shared memory, no coordination overhead.

% Simplified: Alice's process sends to Bob's process

```
send_message(BobPid, Message) ->
    BobPid ! {message, Message},
    receive
        {ack, MessageId} -> {ok, MessageId}
    after 5000 -> {error, timeout}
    end.
```

% Bob's process receives and forwards

```
```

```
handle_message(AlicePid, Message) ->
    store_message(Message),           % save to Mnesia
    deliver_to_client(Message),       % push to Bob's device
    AlicePid ! {ack, Message#msg.id}. % confirm delivery
```

Supervision trees: let it crash

Erlang's OTP (Open Telecom Platform) provides **supervision trees** — a hierarchy of processes where parent processes monitor their children. When a child process crashes (due to a bad message, a network timeout, anything), the supervisor restarts it automatically.

```
Supervisor (never crashes, just restarts)
  ├── Connection Handler 1 (crashed? restart)
  ├── Connection Handler 2 (crashed? restart)
  ├── Message Router (crashed? restart)
  └── Presence Manager (crashed? restart)
```

This is the "let it crash" philosophy: don't write complex error handling code. Let processes fail cleanly, and let supervisors bring them back. The system as a whole stays healthy even as individual components fail and recover. WhatsApp's uptime was legendary precisely because of this design — individual user connection processes could crash and restart without affecting anyone else.

Hot code swapping

Erlang supports hot code swapping. You can deploy new code to a running system without dropping a single connection. For a messaging app where uptime is everything, this meant WhatsApp could push

updates to production servers without disconnecting any of their 2 billion users.

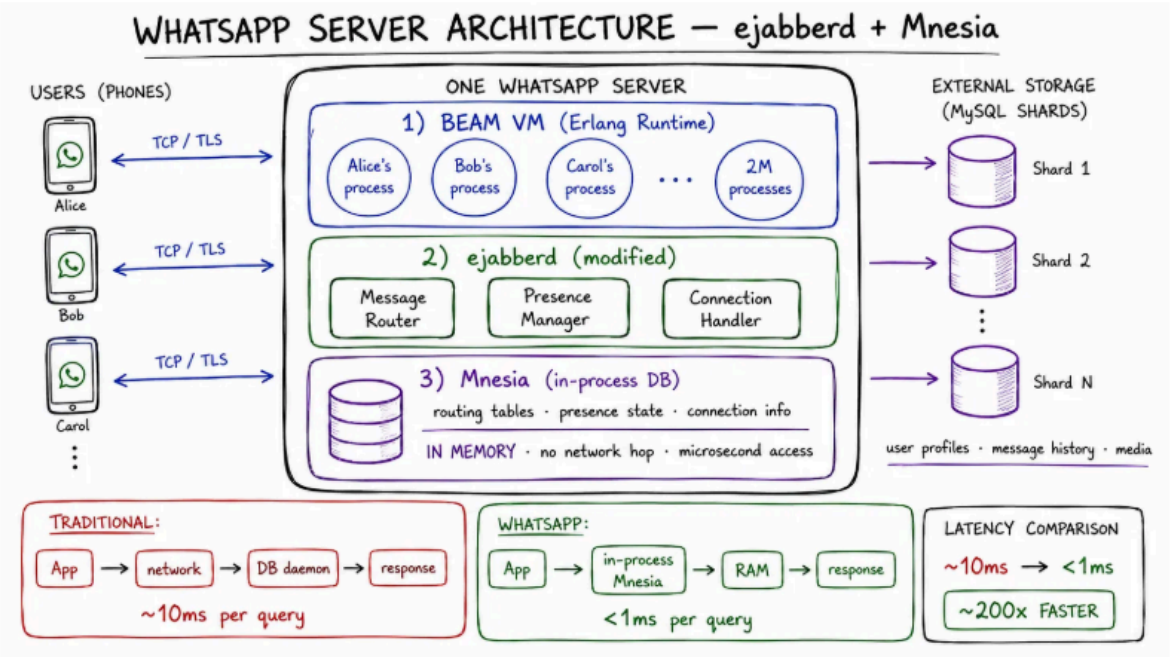
No rolling deploys.

No blue-green deployments. Just swap the code in place.

For a 50-engineer team, this eliminated an entire category of deployment infrastructure. Zero-downtime deploys were a language feature, not a DevOps achievement.

Decision 2: ejabberd — Don't Build From Scratch

WhatsApp didn't build a messaging server from scratch. They built on top of **ejabberd** — an open-source XMPP messaging server written in Erlang, used by thousands of companies worldwide.



They built on top of ejabberd, an open-source XMPP server also written in Erlang — then tailored it to their needs. Instead of building everything in-house, WhatsApp made smart calls. This allowed their small team to focus on performance and reliability, not rebuilding tools.

They did rewrite significant parts of ejabberd's core to meet their performance requirements — connection handling, message routing, presence management. But the fundamental architecture, the protocol logic, and years of battle-tested reliability came for free.

This is the "don't reinvent the wheel" principle in action. A 50-engineer team that tries to build a messaging server from scratch has 50 engineers building plumbing. A team that builds on ejabberd has 50 engineers optimising performance and building user value.

Mnesia: the database that lives inside Erlang

WhatsApp used **Mnesia** — Erlang's built-in distributed database — for routing tables, presence information, and connection state.

Mnesia is not a separate service. It runs inside the same BEAM VM as the application code. Querying it doesn't go over a network socket — it's a function call. This eliminates an entire latency tier:

```
Traditional architecture:
Application → TCP socket → Database daemon → Disk → Response

WhatsApp architecture (for hot data):
Application → in-process Mnesia call → RAM → Response
```

WhatsApp's architecture looked like this: Application (with Mnesia built in). Just Erlang processes on FreeBSD servers. Most modern architectures have

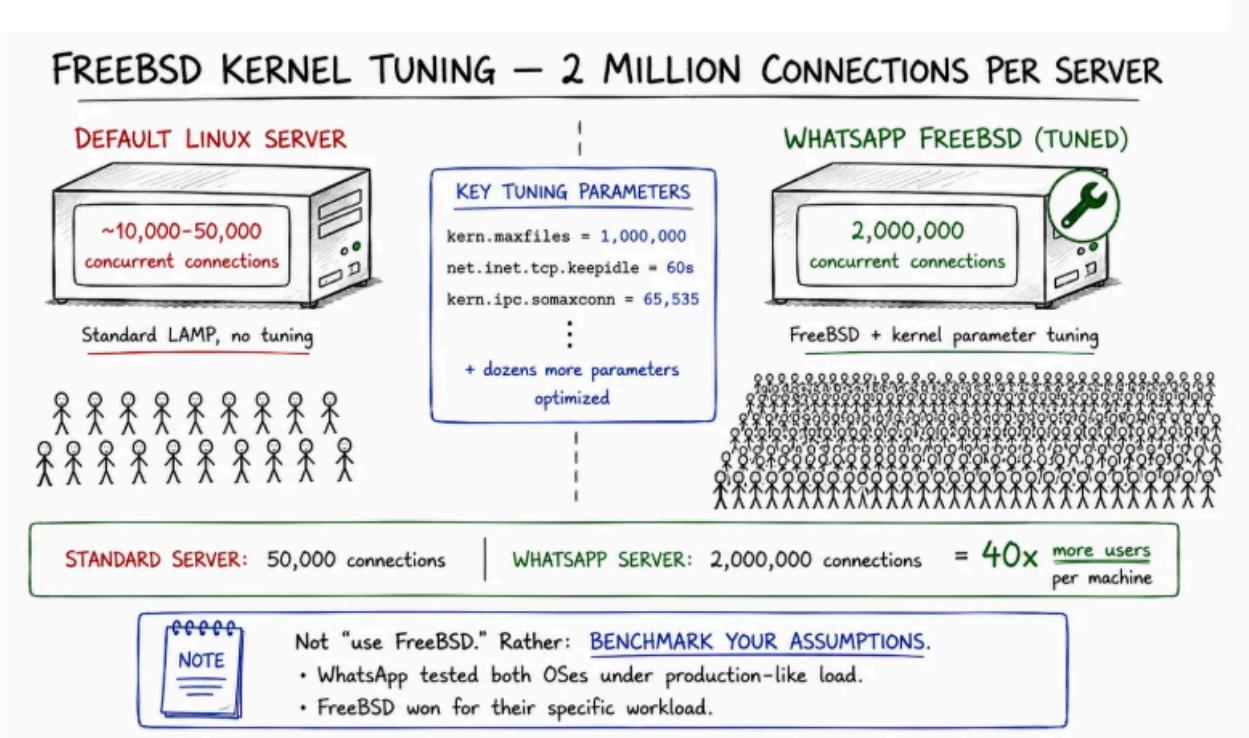
Application → Network → Database Cluster → Disk.

Every one of those arrows is a latency penalty and a failure mode.

For persistent data — user profiles, message history, media — WhatsApp used MySQL shards. Each shard holds data for a subset of users. Mnesia stored only the hot operational data needed for real-time routing.

Decision 3: FreeBSD — Benchmark, Don't Assume

WhatsApp didn't run on Linux. They ran on FreeBSD — a choice that baffled most of the industry. In a world where every startup defaults to Ubuntu, WhatsApp deliberately chose a less popular operating system. The reason came down to one thing: network stack performance. FreeBSD's kqueue event notification system was, at the time, significantly more performant than Linux's epoll for handling massive numbers of concurrent connections.



The WhatsApp founders had previously worked at Yahoo!, where FreeBSD was used extensively. They didn't pick it out of nostalgia — they benchmarked both under realistic load and chose what worked best for their specific workload.

They ran WhatsApp servers on the FreeBSD operating system. Because they had previous experience with FreeBSD while working at Yahoo! Besides FreeBSD offered a reliable network stack. Also they fine-tuned FreeBSD to accommodate 2 million+ connections per server. And modified kernel parameters such as files and sockets.

The kernel tuning was extensive:

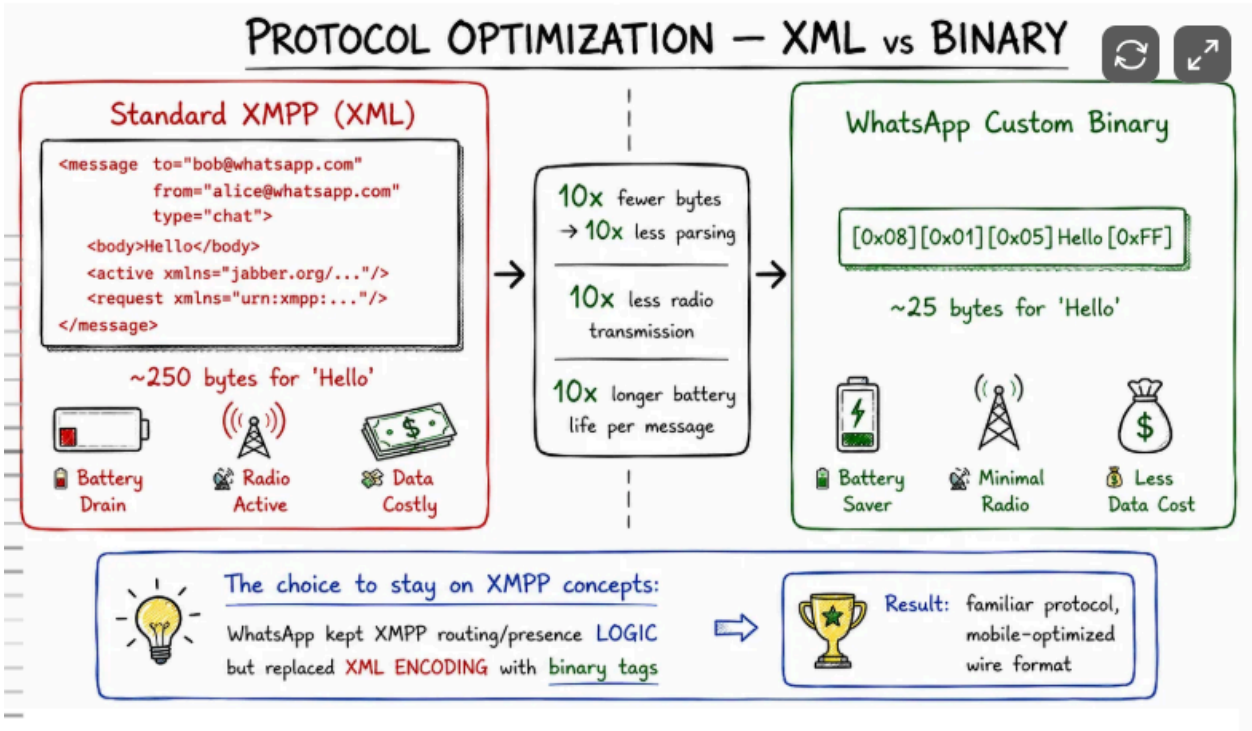
```
# FreeBSD kernel parameters tuned by WhatsApp
# (representative examples — exact values were proprietary)

# File descriptor limits
kern.maxfiles=1000000          # default: 10,000
kern.maxfilesperproc=900000

# Network socket buffers
net.inet.tcp.sendspace=65536
net.inet.tcp.recvspace=65536

# Connection tracking
net.inet.tcp.keepidle=60000    # keepalive interval
net.inet.tcp.keepintvl=5000

# Socket backlog
kern.ipc.somaxconn=65535
```

The result: a single server handling **2 million concurrent TCP connections** — each one a persistent, long-lived connection to an active WhatsApp user.

For comparison, a standard LAMP server without tuning might handle 10,000–50,000 concurrent connections. WhatsApp's tuned FreeBSD servers were 40–200× more efficient per machine.

The lesson here is not "use FreeBSD." It is "test your assumptions." The WhatsApp team benchmarked both operating systems under realistic load and picked what worked best for their specific use case.

Decision 4: The Protocol — Custom Binary Over XMPP XML



XMPP is XML-based. XML is human-readable, self-describing, and carries significant overhead. A simple “hello” message in standard XMPP might look like:

That XML overhead — the tags, the namespaces, the attributes — dwarfs the actual message content. On a mobile network where every byte costs battery and bandwidth, this matters.

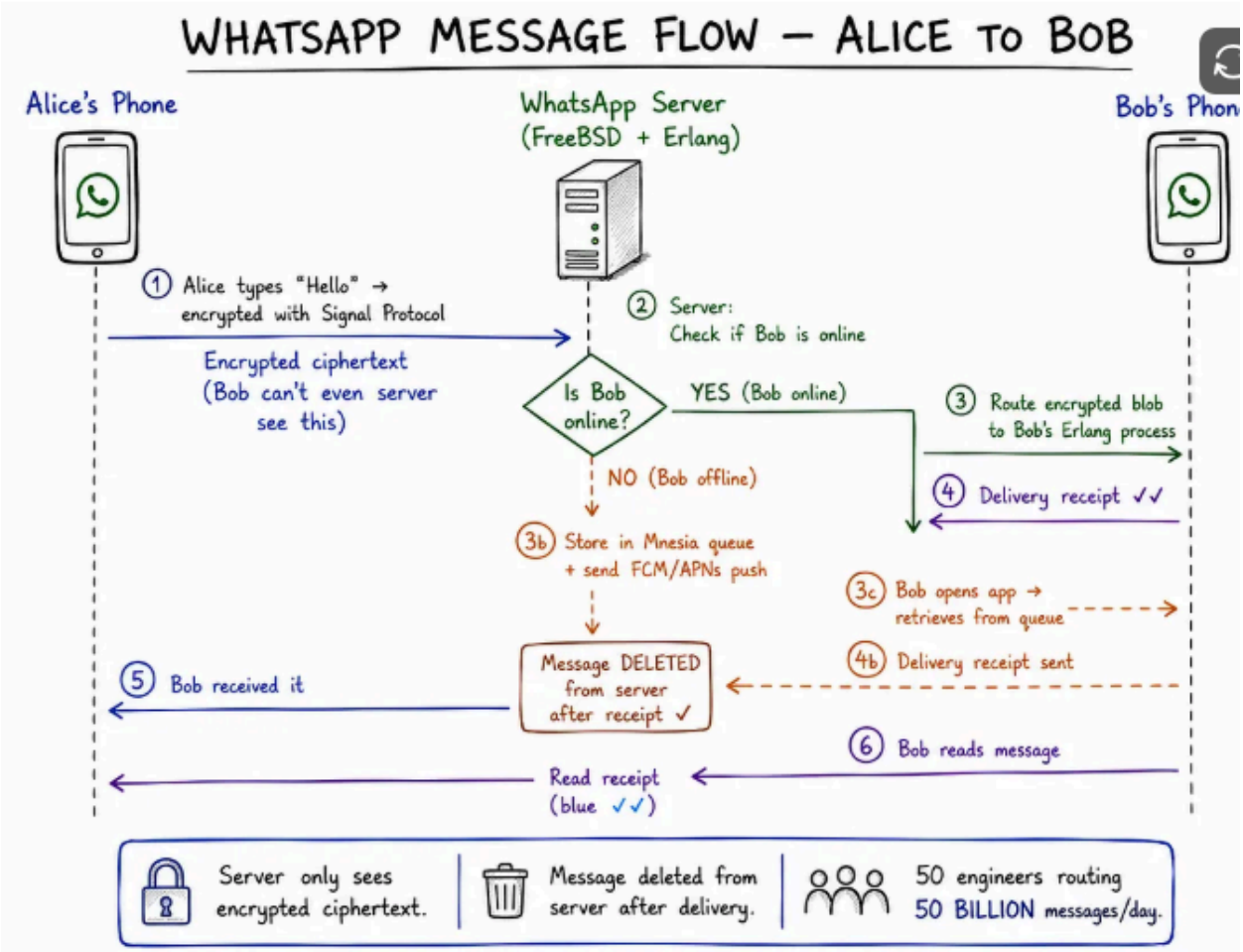
WhatsApp’s wire protocol is a compact, binary, tag-based protocol derived from XMPP concepts but heavily optimized and customized for low bandwidth and mobile devices.

WhatsApp stripped XMPP down to its essential routing logic and replaced XML with a compact binary encoding. The same message in WhatsApp's binary protocol might use 10× fewer bytes. The result:

- **Faster connections** — less data to parse on connection establishment
- **Less data usage** — critical for users in markets with expensive mobile data
- **Longer battery life** — less parsing and less radio transmission time
- **Lower server load** — less CPU spent parsing XML across 2 billion messages

The Message Flow: From Send to Delivered

Here's exactly what happens when **Alice** sends "Hello" to **Bob** on WhatsApp:



Three key design decisions in this flow:

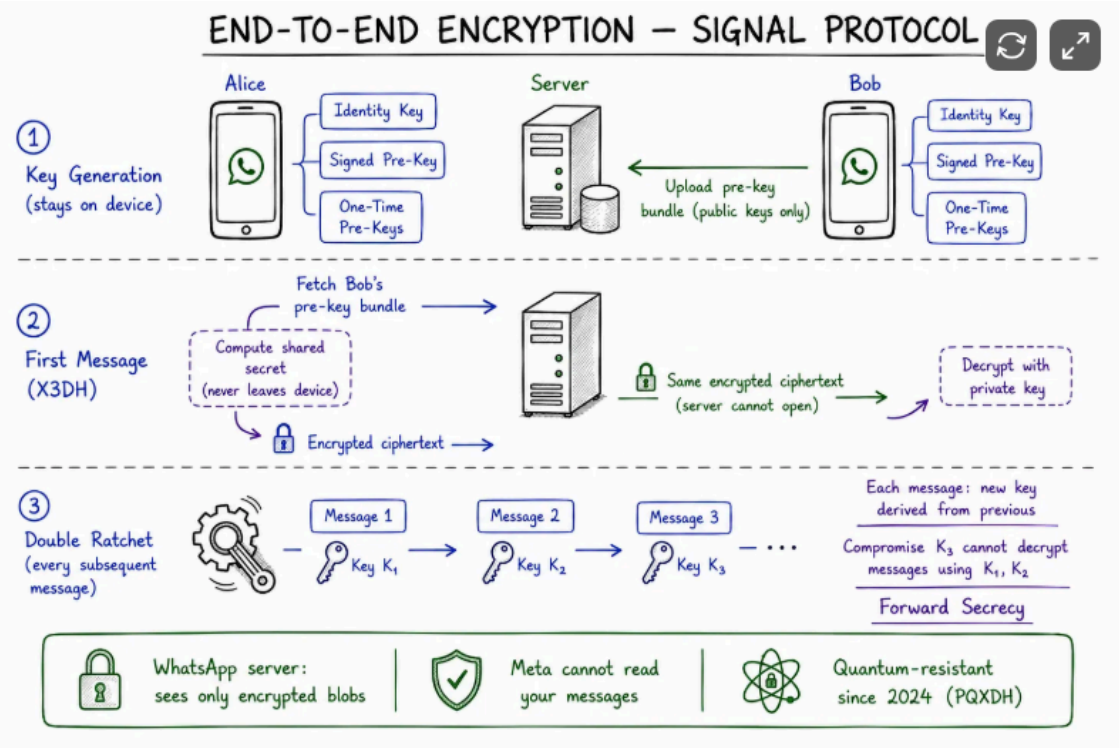
Messages are stored only until delivered. Once Bob's device receives the message and sends an acknowledgement, it's deleted from WhatsApp's servers. This is both a privacy feature and a storage efficiency feature — WhatsApp doesn't store the world's chat history.

The server only sees ciphertext. Since 2016, all messages are end-to-end encrypted with the Signal Protocol. WhatsApp's servers cannot read message content. They're essentially routing boxes for encrypted blobs.

Delivery receipts drive the protocol. In prevalent end-to-end encrypted messaging protocols used in WhatsApp and Signal, delivery receipts indicate the successful decryption of a message and thus allow the sender to mark the transmitted message and the associated ephemeral keying material for deletion.

The tick system (✓ sent, ✓✓ delivered, ✓✓ blue = read) is the visible surface of a cryptographic acknowledgement protocol.

Decision 5: End-to-End Encryption with Signal Protocol



In April 2016, WhatsApp rolled out end-to-end encryption for all messages, calls, photos, and videos. The implementation uses the **Signal Protocol** — developed by Open Whisper Systems, the same protocol used by the Signal app.

WhatsApp uses the Signal Protocol for end-to-end encryption on 1:1 chats, group chats, voice and video calls. Only the sender and recipient devices can read message content; the server stores only ciphertext. Meta cannot read the content of private chats.

The Signal Protocol combines several cryptographic primitives:

X3DH (Extended Triple Diffie-Hellman): Used on initial key exchange. When you first message someone, your devices perform a Diffie-Hellman

key agreement that produces a shared secret — without ever transmitting the secret itself over the network.

Double Ratchet Algorithm: After the initial handshake, each message uses a new encryption key derived from the previous one. This provides **forward secrecy** — if an attacker captures your encrypted messages today and later steals your current keys, they still can't decrypt past messages because those used different keys that are now gone.

Pre-keys: Since WhatsApp users are often offline, the Signal Protocol uses "pre-keys" — a bundle of public keys uploaded to WhatsApp's servers that allow Alice to start encrypting messages to Bob even when Bob is offline, without a real-time key exchange.

Key generation (**happens on device, never leaves device**)**:**

- Identity Key Pair (permanent)
- Signed Pre-Key (rotated regularly)

One-Time Pre-Keys (batch of 100, replenished as used)

When Alice messages Bob for the first time:
1. Alice fetches Bob's pre-key bundle from server
2. X3DH: Alice + Bob's keys → shared secret (on Alice's device)
3. Message encrypted with derived key
4. Sent to server (server sees only ciphertext)
5. Bob decrypts with his private key
6. Double Ratchet: both devices advance their key state

Result:

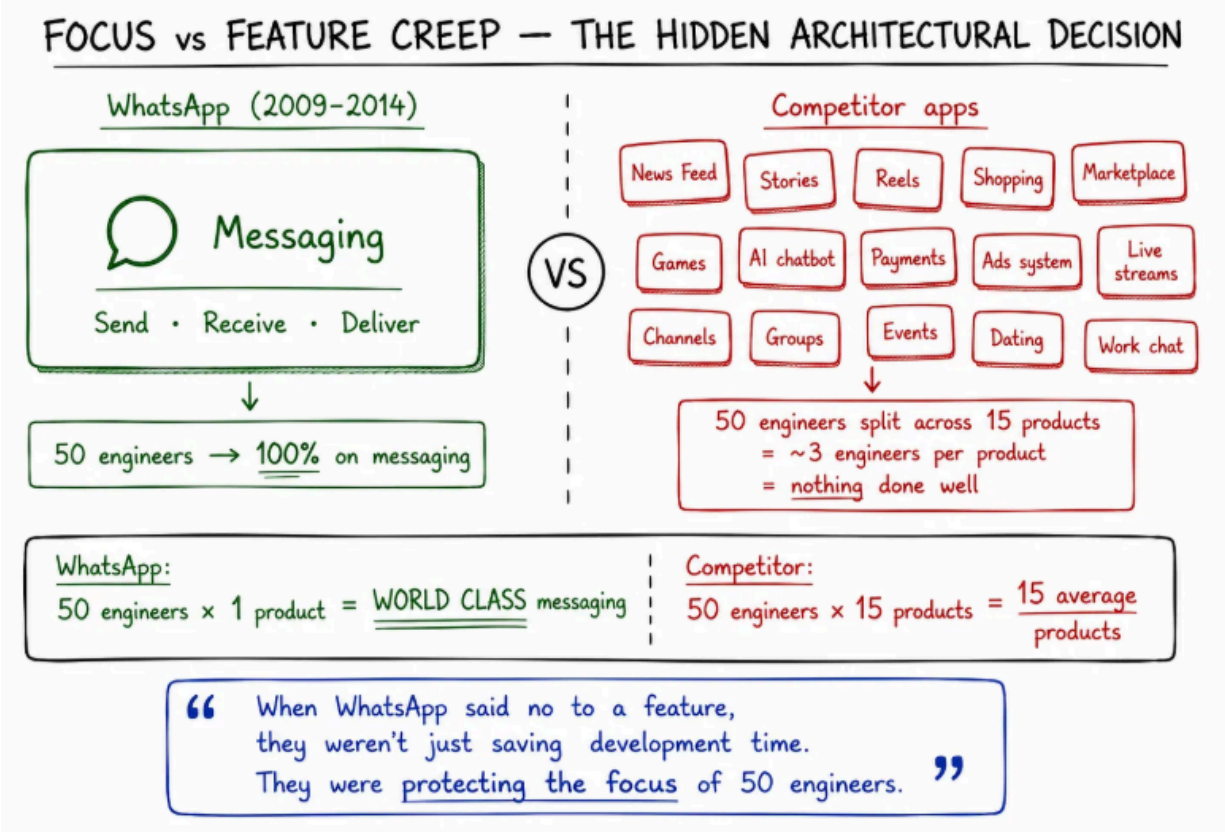
Every message uses a different encryption key.

Compromise of today's keys ≠ decrypt yesterday's messages.

Since 2024, WhatsApp has also added **post-quantum key exchange** (**PQXDH**) — protecting encrypted messages against future quantum computers that might break current elliptic-curve cryptography.

Decision 6: Do One Thing Extremely Well

For years, WhatsApp had exactly one feature: send and receive messages. No stories. No channels. No payment systems. No games. No shopping tabs. No AI chatbots. No disappearing messages. No status updates. Just messaging.



This is the most underrated technical decision WhatsApp made. Every feature you don't build is:

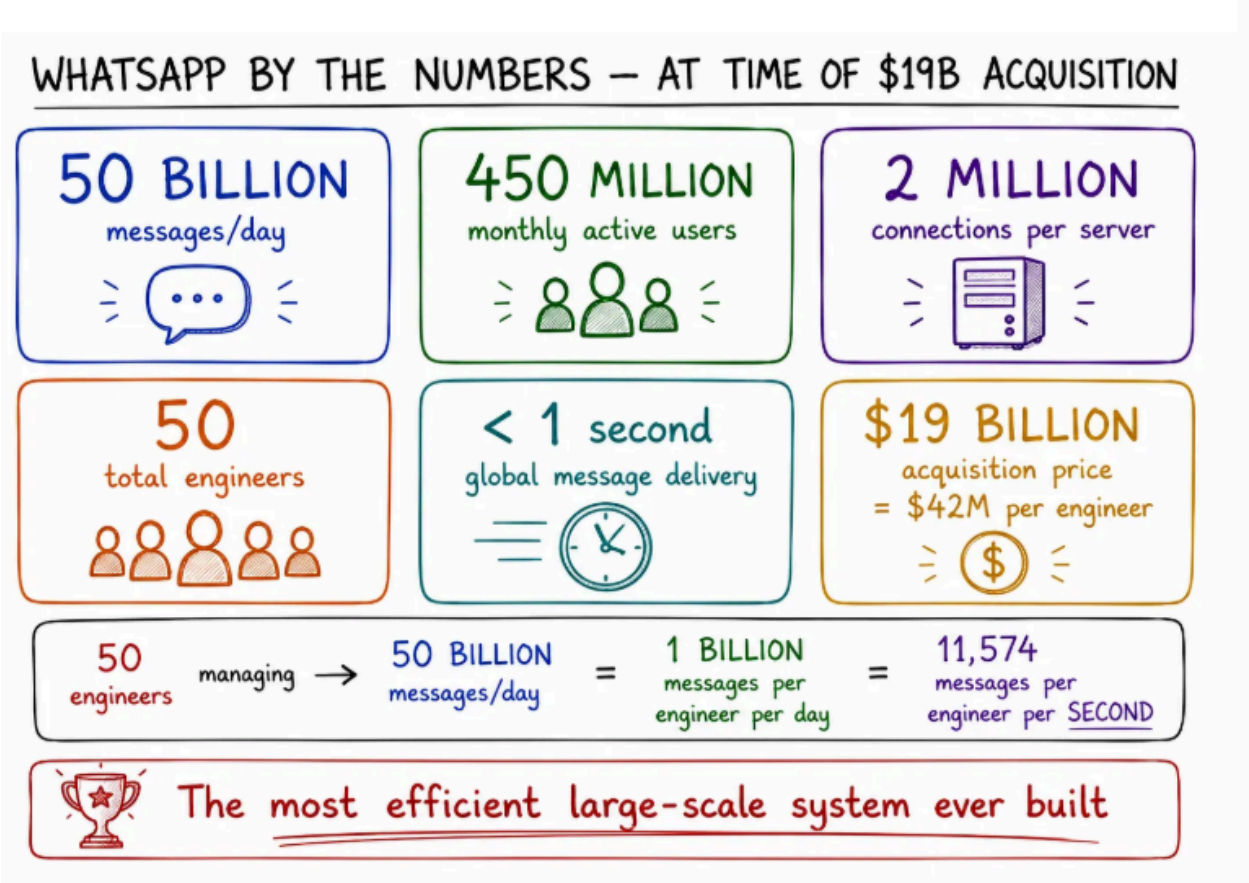
- Infrastructure you don't provision
- Code you don't write, debug, or maintain
- An attack surface that doesn't exist
- Engineering bandwidth that goes to your core product instead

A news feed requires a recommendation algorithm, content moderation, an ads system, and a publisher API. WhatsApp had none of these. Their 50 engineers worked only on making messaging faster, more reliable, and more private.

The compound effect: each “no” to a feature meant more headroom per engineer for the core product. WhatsApp’s message delivery latency, reliability, and battery efficiency were best-in-class precisely because all 50 engineers could obsess over those metrics instead of splitting attention across dozens of product surfaces.

The Numbers: What 50 Engineers Actually Ran

By the time of the Facebook acquisition in 2014, WhatsApp’s 50 engineers were running:

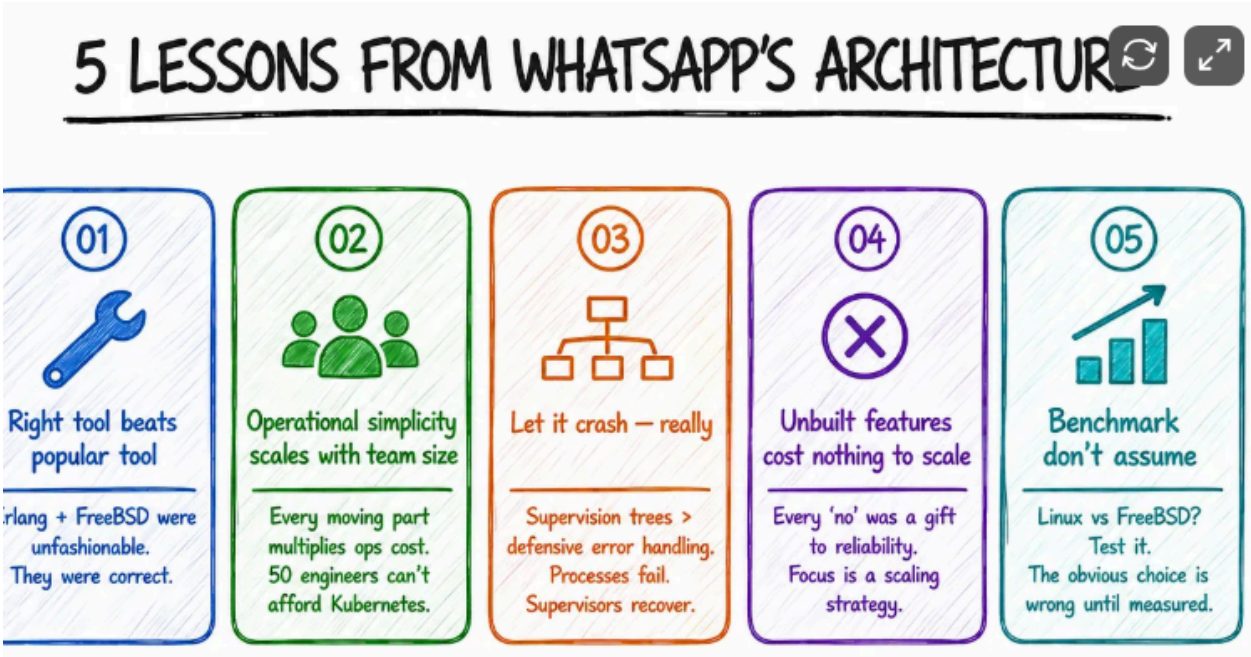


Daily messages processed:	50 billion
Monthly active users:	450 million
Daily active users:	~320 million
Concurrent connections:	~500 million peak
Connections per server:	2 million
Servers in operation:	~hundreds (not thousands)
Message delivery latency:	< 1 second (global)
Uptime:	99.9%+
Engineers per 1M users:	0.11 (industry: ~1-10)

The economics were extraordinary. At Twitter's engineer-to-user ratio, WhatsApp would have needed 4,500 engineers. At a loaded salary cost of

$300K per engineer, that's a $1.35 billion annual engineering bill vs WhatsApp's actual $15 million. Facebook's $19 billion acquisition price looked different knowing they were buying technology that was 100× more operationally efficient than the alternative.

What Engineers Should Take From This



1. Right tool beats popular tool

Erlang has a tiny ecosystem. FreeBSD has a small community. Neither choice was defensible on "hiring pipeline" grounds. Both were defensible on "solves our specific problem better" grounds.

WhatsApp chose Erlang when everyone else used Java or Python. They chose FreeBSD when everyone used Linux. These

unconventional choices matched their specific requirements. Do not pick technologies because they are popular. Pick them because they solve your actual problems.

2. Operational simplicity scales with team size

Every moving part in your architecture is something your team must understand, deploy, monitor, debug, and upgrade. At 50 engineers, this cost is enormous. At 500 engineers, it's manageable. The architectural decisions that look "sub-optimal" to a large team (in-process Mnesia instead of separate database, FreeBSD instead of Kubernetes) were precisely right for a small team.

3. "Let it crash" is a real architecture

Erlang's supervision trees implement a failure model that most systems paper over with complex error handling. Instead of trying to anticipate every failure mode and write recovery code, WhatsApp let processes fail and trusted supervisors to restart them. The result was more resilient than any defensive programming approach.

4. The feature you don't build costs nothing to scale

Every WhatsApp feature that didn't exist in 2014 was a feature that required zero engineering headroom, zero infrastructure, zero on-call rotation, zero scaling decisions. Product focus is a scalability strategy.

5. Benchmark, don't assume

Linux or FreeBSD? Java or Erlang? Shared-nothing or shared-memory? WhatsApp tested their assumptions under realistic load and chose based

on evidence. The “obvious” choice is obvious to people who haven’t measured. The right choice is obvious after you measure.

Too Long; Didn’t Read

- **50 engineers, 450M users, 50B messages/day** — the most efficient large-scale system ever built
- **Erlang actor model** — lightweight processes (~300 bytes each vs 1-8MB OS threads), 2M connections/server, direct process-to-process message delivery
- **OTP supervision trees** — “let it crash” philosophy, supervisors restart failed processes, system stays healthy
- **Hot code swapping** — deploy new code without dropping a single connection, no rolling restarts
- **ejabberd** — built on proven open-source XMPP server, rewrote core components, didn’t reinvent from scratch
- **Mnesia** — Erlang’s in-process distributed DB for routing tables and presence, eliminates network hop to database
- **FreeBSD** — benchmarked vs Linux, kqueue outperformed epoll for persistent connections, kernel tuned for 2M connections/server
- **Custom binary XMPP** — replaced XML with compact binary encoding, 10× fewer bytes per message, better battery life
- **Message flow** — stored only until delivered, server sees only ciphertext, tick system (✓ sent ✓✓ delivered ✓✓ read) is cryptographic receipt protocol
- **Signal Protocol** — X3DH initial handshake, Double Ratchet for forward secrecy, pre-keys for offline messaging, post-quantum (PQXDH) since 2024

- **One product** — messaging only, no features, all 50 engineers focused on delivery latency and reliability

---

If you want more articles like this let us know , we the team of loop are ready to contribute more ……..